

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)

Activity Tool: Comparative Analysis (Low)

Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.	Bloom's Level: 4 – Analyze
2	Differentiate between clustered indexing and non-clustered indexing with suitable database examples.	Bloom's Level: 4 – Analyze
3	Compare primary indexing and secondary indexing based on storage requirements and search performance.	Bloom's Level: 4 – Analyze
4	Analyze the differences between static hashing and dynamic hashing in database systems.	Bloom's Level: 4 – Analyze
5	Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.	Bloom's Level: 4 – Analyze
6	Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.	Bloom's Level: 4 – Analyze
7	Compare dense indexing and sparse indexing with respect to memory usage and access speed.	Bloom's Level: 4 – Analyze
8	Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.	Bloom's Level: 4 – Analyze
9	Compare single-level indexing and multi-level indexing for efficient database retrieval operations.	Bloom's Level: 4 – Analyze
10	Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.	Bloom's Level: 5 – Evaluate

Code of Conduct:

I SANTHOSH E (192511470) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

E.Santhosh

Q1. Compare Sequential File Organization and Heap File Organization

Sequential file organization stores records in a particular order, usually according to a primary key. Heap file organization stores records in no particular order.

Comparison

Sequential File: Records are sorted/ordered, insertion is slow or moderate, search is efficient for ordered data, and range queries are very efficient.

Heap File: Records are unordered, insertion is very fast, search generally requires a linear scan, and range queries are inefficient.

Insertion

In a heap file, a new record can be placed in any available location. In a sequential file, the correct sorted position must be maintained, which may require shifting records or reorganizing blocks.

Deletion

Heap files can mark a record as deleted or reuse its space. Sequential files may require additional maintenance to preserve ordering.

Retrieval

Sequential files can use binary search or an index for efficient retrieval. Heap files without an index may require $O(N)$ record or block examination in the worst case.

Example

Sequential: 1001 $\rightarrow$ 1002 $\rightarrow$ 1003 $\rightarrow$ 1004 $\rightarrow$ 1005

Heap: 1003 $\rightarrow$ 1001 $\rightarrow$ 1005 $\rightarrow$ 1002 $\rightarrow$ 1004

Conclusion

Sequential files are better for ordered and range retrieval, while heap files are better when frequent insertion and simple storage are the main requirements.

Q2. Differentiate Clustered Indexing and Non-Clustered Indexing

A clustered index determines the physical/logical ordering of records, while a non-clustered index is a separate structure containing keys and pointers to the actual records.

Clustered Index

The records are stored according to the indexed order. Example:

Student_ID $\rightarrow$ Student Records

1001 → Record 1
1002 → Record 2
1003 → Record 3

Non-Clustered Index

The index contains pointers to records. Example:

Department
CSE → [Address 500, Address 900, Address 1200]
ECE → [Address 600, Address 1100]

Comparison

Clustered: follows data order, usually one main clustered order, excellent for range queries, and has relatively low additional storage overhead.

Non-clustered: separate from data order, multiple indexes can be created, useful for additional search attributes, and requires extra index storage.

Database Example

```
CREATE INDEX idx_student_id ON Student(Student_ID);  
  
CREATE INDEX idx_department ON Student(Department);
```

Conclusion

Use clustered indexing when records are frequently retrieved in a particular order or by ranges. Use non-clustered indexes for additional search attributes.

Q3. Compare Primary Indexing and Secondary Indexing

A primary index is created on the ordering/primary key of a sorted file. A secondary index is created on a non-ordering attribute to support additional search operations.

Primary Index

If student records are sorted by Student_ID:

Student_ID → Address
1001 → 500
1002 → 600
1003 → 700

A primary index can often be sparse because it can point to blocks rather than every record.

Secondary Index

If records are ordered by Student_ID but searches are also required by Department:

CSE → 1001, 1005, 1010
ECE → 1002, 1006

Secondary indexes are commonly dense because several records may share the same search value.

Comparison

Primary Index: uses the ordering/primary key, usually unique, requires relatively less storage, and is efficient for primary-key searches.

Secondary Index: uses a non-ordering attribute, may contain duplicate values, requires more storage, and supports additional search patterns.

Conclusion

Primary indexing is efficient and space-saving for ordered primary-key access, while secondary indexing provides fast access through additional attributes at the cost of extra storage.

Q4. Analyze Static Hashing and Dynamic Hashing

Hashing maps a search key to a bucket using a hash function such as $h(\text{key}) = \text{key} \% M$.

Static Hashing

In static hashing, the number of buckets remains fixed. If the file grows, overflow buckets may be required.

Bucket 0 | Bucket 1 | Bucket 2 | Bucket 3

↓
Overflow

Dynamic Hashing

Dynamic hashing allows buckets to expand or split as the database grows. Common examples are extendible hashing and linear hashing.

Comparison

Static hashing: fixed buckets, simple implementation, more overflow when the database grows, and limited scalability.

Dynamic hashing: changing number of buckets, better overflow control, more complex implementation, and high scalability.

Example

If $h(\text{ID}) = \text{ID} \% 4$ and bucket 2 becomes full, static hashing may use an overflow bucket. Dynamic hashing can split the full bucket and redistribute its records.

Conclusion

Static hashing is simple and suitable when database size is predictable. Dynamic hashing is better when records are frequently inserted or the database grows significantly.

Q5. Compare B-Tree and B+ Tree Indexing for Large-Scale Databases

B-Tree and B+ Tree are balanced tree indexing techniques used for efficient searching.

B-Tree

In a B-Tree, keys and record pointers can occur in both internal nodes and leaf nodes.

B+ Tree

In a B+ Tree, internal nodes contain search keys, while actual record pointers are stored at leaf nodes. Leaf nodes are linked for sequential and range access.

Comparison

B-Tree: data pointers may occur in internal and leaf nodes, usually has lower fan-out, and may have greater height.

B+ Tree: data pointers are stored at leaves, has higher fan-out, generally lower height, and provides excellent range and sequential access.

Complexity

Both provide approximately $O(\log N)$ search complexity. B+ Trees generally have higher fan-out, resulting in fewer disk accesses.

Example

For a range query from 20 to 50, linked B+ Tree leaves can be traversed as:

20 → 25 → 30 → 35 → 40 → 45 → 50

Conclusion

For large-scale database applications, B+ Tree is generally preferred because of high fan-out, lower tree height, linked leaves, and efficient range retrieval.

Q6. Differentiate Linear Hashing and Extendible Hashing

Both are dynamic hashing techniques designed to handle growing databases without requiring a complete reorganization.

Linear Hashing

Linear hashing gradually splits buckets one at a time. It does not require a large directory.

Extendible Hashing

Extendible hashing uses a directory containing pointers to buckets. When a bucket overflows, it splits; if necessary, the directory doubles.

Comparison

Linear Hashing: no directory is required, bucket splitting is gradual, memory overhead is lower, and growth is smooth.

Extendible Hashing: uses a directory, splitting depends on overflow and local depth, directory doubling may occur, memory overhead is higher, and growth is controlled.

Conclusion

Linear hashing is useful when gradual growth and lower directory overhead are important. Extendible hashing is suitable when fast dynamic expansion and controlled bucket access are required.

Q7. Compare Dense Indexing and Sparse Indexing

A dense index contains an index entry for every search-key value or record, while a sparse index contains entries only for selected records or blocks.

Dense Index

Example:

1001 → Record 1

1002 → Record 2

1003 → Record 3

1004 → Record 4

Advantages: very fast search and direct record location. Disadvantage: higher memory and storage requirement.

Sparse Index

Example:

1001 → Block 1

1051 → Block 2

1101 → Block 3

The database locates the nearest index entry and then searches within the corresponding data block.

Comparison

Dense indexing uses more entries and storage but provides faster search. Sparse indexing uses fewer entries and less storage but may require an additional search within a block.

Conclusion

Dense indexing provides faster access but uses more memory. Sparse indexing saves storage and is commonly suitable for ordered primary files.

Q8. Indexed Sequential File Organization vs Direct File Organization

Indexed Sequential File Organization stores records sequentially and maintains an index. Direct file organization uses hashing or direct addressing to locate records.

Indexed Sequential

Index → Block 1 → Records

→ Block 2 → Records

→ Block 3 → Records

It supports both sequential and direct access and is efficient for range queries.

Direct File

Key → Hash Function → Bucket → Record

It provides very fast exact-match retrieval but does not preserve ordering.

Comparison

Indexed Sequential: good exact search, excellent range search, excellent sequential processing, moderate insertion cost, and ordered access.

Direct: excellent exact search, poor range search, fast insertion, and unordered access.

Advantages and Limitations

Indexed sequential organization supports ordered processing and range queries but requires index maintenance and may need overflow areas.

Direct organization provides fast exact-match access but is inefficient for range queries.

Conclusion

Use indexed sequential organization when both sequential/range access and direct lookup are needed. Use direct organization when exact-match retrieval is the primary requirement.

Q9. Compare Single-Level and Multi-Level Indexing

Single-level indexing uses one index to locate records. Multi-level indexing creates an index on the index, reducing disk accesses for large databases.

Single-Level

Search Key → Index → Data Block

Multi-Level

Search Key → Level 1 Index → Level 2 Index → Data Block

Comparison

Single-level indexing has one index, lower storage overhead, and is suitable for smaller indexes.

Multi-level indexing has multiple levels, slightly higher index storage, better scalability, and reduced disk I/O for large files.

Complexity

Multi-level indexes reduce the search space at each level. For a B+ Tree-like structure, search is approximately $O(\log_B N)$, where B is the branching factor.

Conclusion

Multi-level indexing is preferable for large databases because it reduces disk I/O and provides faster record retrieval.

Q10. Evaluate Hashing and Indexing for Exact-Match and Range Queries

Hashing and indexing are optimized for different types of queries.

Exact-Match Query

Example:

```
SELECT * FROM Student  
WHERE Student_ID = 1001;
```

Hashing is highly efficient for exact-match queries. Average lookup complexity is $O(1)$ when collisions are well controlled.

Range Query

Example:

```
SELECT * FROM Student  
WHERE CGPA BETWEEN 8.0 AND 9.0;
```

A B+ Tree is more suitable because it preserves key ordering and its linked leaves support efficient range traversal.

Comparison

Hashing: excellent for exact match, poor for range queries, average exact search $O(1)$, and does not provide ordered retrieval.

B+ Tree: excellent for exact and range searches, approximately $O(\log N)$ search, and supports ordered/sequential access.

Performance Evaluation

For exact-match queries: Hashing is generally faster on average.

For range queries: B+ Tree is better because it preserves ordering.

Practical Recommendation

For a database requiring both types of access, hashing can be used for exact-match keys and B+ Tree indexes for range/date/value fields. The final design should depend on workload, update frequency, memory, and storage requirements.

Conclusion

Hashing is best for exact-match queries, while B+ Tree indexing is best for range and ordered queries. A combination can provide excellent overall database performance.

Quick Revision Table

Q.No	Topic	Key Point
1	Sequential vs Heap	Sequential = ordered/range; Heap = fast insertion
2	Clustered vs Non-clustered	Clustered follows data order; non-clustered is separate
3	Primary vs Secondary	Primary uses ordering key; secondary supports other attributes
4	Static vs Dynamic Hashing	Dynamic hashing handles database growth
5	B-Tree vs B+ Tree	B+ Tree preferred for large DBs and range queries
6	Linear vs Extendible Hashing	Both are dynamic; extendible hashing uses a directory
7	Dense vs Sparse Index	Dense = more entries/faster; sparse = less storage
8	Indexed Sequential vs Direct	ISAM supports range/sequential; direct is strong for exact lookup
9	Single vs Multi-Level	Multi-level indexing reduces disk I/O
10	Hashing vs Indexing	Hashing for exact match; B+ Tree for ranges

Important Terms and Formulas

- Hash function: $h(\text{key}) = \text{key} \% M$
- Hash search – average case: $O(1)$
- Binary/B+ Tree search: approximately $O(\log N)$
- Heap sequential search: $O(N)$
- Range queries are generally better supported by ordered indexes such as B+ Trees.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand